hexagone
École Supérieure d'Informatique

Projet de groupe

1GEST – Gestion de projet et outils collaboratifs · 12 et 16 novembre 2026

Contexte

Votre équipe vient d'être choisie par un client pour réaliser une application. Le client sait ce qu'il veut… à peu près. Son besoin est flou, il oublie des détails importants et il changera peut-être d'avis en cours de route. Comme dans un vrai projet !

Votre mission : **comprendre son besoin, le découper, l'estimer, le planifier et suivre le projet** avec les outils d'une vraie équipe. Vous n'écrirez aucune ligne de code. Tout ce que vous produisez, ce sont les documents et les outils qui permettraient à une équipe de développement de démarrer lundi matin.

Ce projet en bref

- **Équipes** de 3. Le groupe de Clermont-Ferrand travaille ensemble au campus et me questionne sur Teams.
- **Atelier 1** (jeudi 12 novembre) : cadrage, backlog, estimation et tableau Trello.
- **Atelier 2** (lundi 16 novembre seulement) : le projet passe sur GitHub, avec les documents de l'équipe.
- **Soutenance** de 10 minutes le lundi 16 novembre à partir de 14h.
- Le projet compte pour **50 %** de la note du module (livrables sur 12, soutenance sur 8).

Groupe de Clermont-Ferrand

- Vous travaillez **ensemble dans la même salle**, comme les équipes de Versailles. Toutes les étapes se font en face à face.
- Un ordinateur reste connecté à la réunion Teams, micro coupé quand vous travaillez. Allumez-le pour me parler.
- Je vous rejoins sur Teams dès que vous le demandez. Pour me montrer votre travail, partagez l'écran de l'ordinateur concerné.
- Une question ? Écrivez-la dans le chat de la réunion ou interpellez-moi en ouvrant votre micro, je réponds dès que possible.
- Si la connexion coupe, continuez avec cet énoncé et reconnectez-vous dès que possible. Rien n'est perdu.

Atelier 1 – Simuler un projet

Chaque étape se termine par un encadré « Pour valider, assurez-vous que… ». Vous n'avez pas à me le montrer tout de suite : je vérifie ces points pendant le point d'étape de fin de journée.

1 L'équipe et les rôles

Acteurs · Scrum

Formez votre équipe de trois puis répartissez ces rôles. Chaque rôle est tenu par une seule personne. Le Product Owner et le Scrum Master sont aussi les Developers de l'équipe.

Rôle	Ce qu'il fait pendant le projet
Client (MOA)	Reçoit le cahier des charges et il est le **seul** à le lire. Il exprime le besoin, répond aux questions et valide les livrables. Il joue son personnage jusqu'au bout.
Product Owner	Représente le client dans l'équipe. Il rédige et priorise le backlog. Il a le dernier mot sur les priorités.
Scrum Master	Anime les étapes, surveille le temps et tient le tableau à jour. Il crée le dépôt GitHub de l'équipe.

Tout le monde, client compris, rédige une partie des documents de l'atelier 2.

Dès que les rôles sont choisis, le client me le signale. Je lui envoie son cahier des charges en privé, dans un message Teams, Wimi ou par e-mail.

> **Pour valider, assurez-vous que…**
>
> - chacun connaît son rôle ;
> - le client a reçu son cahier des charges.

2 Le besoin du client

QQOQCCP · Silent Scrum

Vous allez découvrir le besoin avec un jeu en trois manches, le **Silent Scrum**. Il oblige à écouter avant de parler.

1. **Le client parle, l'équipe se tait (5 minutes).** Le client présente son projet avec ses mots, sans lire son cahier des charges. L'équipe écoute et prend des notes, sans poser la moindre question ni réagir.
2. **L'équipe pose ses questions (15 minutes).** Elle s'appuie sur le QQOQCCP (Quoi, Qui, Où, Quand, Comment, Combien, Pourquoi). Le client répond **uniquement** à la question posée. Ce qu'on ne lui demande pas, il ne le dit pas.

3. **L'équipe reformule (10 minutes).** Le Product Owner résume le besoin en cinq phrases maximum. Le client dit ce qui est juste, ce qui est faux et ce qui manque. On recommence jusqu'à ce qu'il valide.

Le Scrum Master chronomètre chaque manche. Une fois le temps écoulé, on passe à la suivante : le client devra préciser le reste plus tard, comme dans un vrai projet.

Notez le résultat : un objectif SMART, les utilisateurs, les livrables attendus, les jalons, les contraintes et ce qui est **hors périmètre**.

Choisissez enfin un **nom de projet** court et cohérent avec le besoin, validé par le client. Il servira pour le tableau Trello et le dépôt GitHub.

Des idées de questions à poser au client

- Qui utilisera l'application ? Sur quel appareil ? Où ?
- Pour quelle date en avez-vous besoin ? Pourquoi cette date ?
- Quelle fonctionnalité est indispensable dès le premier jour ?
- Qu'est-ce qui se passe aujourd'hui, sans l'application ?
- Y a-t-il quelque chose que vous ne voulez surtout pas ?

> **Pour valider, assurez-vous que…**
>
> - votre client a validé la reformulation du besoin ;
> - vous avez noté au moins un élément hors périmètre ;
> - le projet a un nom.

3 Le backlog

User stories · MoSCoW

Le Product Owner rédige le backlog avec l'équipe. Il contient **au moins 10 user stories**, chacune avec :

- la phrase « En tant que…, je veux… afin de… » ;
- au moins deux **critères d'acceptation** ;
- une priorité **MoSCoW** (Must, Should, Could, Won't), validée par le client.

Exemple d'une story bien écrite pour un autre projet :

> **En tant qu'**arbitre du tournoi, **je veux** saisir le résultat d'une course depuis mon téléphone **afin de** mettre à jour le classement sans attendre la fin du tournoi.
>
> - Je choisis la course puis la position de chaque joueur.
> - Le classement est mis à jour en moins de 10 secondes.
>
> *Priorité : Must*

Pour valider, assurez-vous que...

- le backlog compte 10 user stories ou plus, bien formées, avec leurs critères d'acceptation ;
- chaque story a une priorité et au moins une est en Won't.

4 L'estimation et les sprints

Planning Poker · jalons

Estimez chaque story en **points** avec un Planning Poker. Le client n'estime pas : il répond aux questions.

Chaque membre qui estime installe une application gratuite de Planning Poker sur son téléphone :

- **Planning-Poker**, gratuite et sans publicité : iPhone ou Android ;
- sans rien installer, depuis le navigateur du téléphone : PlanITpoker.

1. Le Product Owner lit la story.
2. Chacun choisit en secret une valeur parmi **1, 2, 3, 5, 8, 13** ou « ? ».
3. Au signal, tout le monde révèle sa valeur en même temps.
4. Les deux valeurs extrêmes s'expliquent. On revote jusqu'à se mettre d'accord.

Une story estimée à 13 est trop grosse : découpez-la en plusieurs stories plus petites.

Planifiez ensuite **deux sprints de deux semaines**, avec une capacité de **20 points par sprint** :

- le sprint 1 ne contient que des Must ;
- chaque fin de sprint est un **jalon** daté, avec ce que le client pourra voir ce jour-là ;
- ce qui ne tient pas dans les deux sprints reste dans le backlog. Dites-le au client !

Prenez le lundi 30 novembre 2026 comme date de début du sprint 1.

Pour valider, assurez-vous que...

- chaque story a une estimation en points ;
- chacun des deux sprints compte 20 points au maximum ;
- les deux jalons sont datés.

5 Le tableau Trello

Kanban · Trello

1. Le Scrum Master crée un compte Trello (avec la double authentification) puis un tableau qui porte le nom du projet.
2. Il invite tous les membres et moi-même : **chris.chevalier@ecole-hexagone.com**.
3. Créez cinq listes : **Backlog**, **Sprint 1 – À faire**, **En cours**, **En revue**, **Terminé**.
4. Une carte par user story, avec :
   - le titre court de la story et la phrase complète dans la description ;
   - une étiquette de priorité (Must, Should, Could, Won't) et les points dans le titre, par exemple « Saisie des scores (5) » ;
   - une checklist avec les critères d'acceptation.
5. Rangez les stories du sprint 1 dans « Sprint 1 – À faire ». Les autres restent dans « Backlog ».

> **Pour valider, assurez-vous que…**
>
> - le tableau est partagé avec toute l'équipe et avec moi ;
> - chaque carte est complète : description, étiquette, points, checklist.

6 Le point d'étape

Reporting

En **fin d'après-midi**, chaque équipe présente son avancement en **3 minutes**, en partageant son tableau Trello. Ce point n'est pas noté. J'en profite pour vérifier les encadrés « Pour valider » des étapes 1 à 5.

- Le projet en une phrase et la météo du projet : ☀️ dans les temps, ⛅ sous surveillance, ⛈️ en difficulté.
- Ce qui est fait.
- Ce qui reste à faire.
- Une difficulté rencontrée et ce que vous attendez de moi ou du client.

⌂ Avant le lundi 16 novembre

Travail personnel

Chacun, seul, si ce n'est pas déjà fait en séance :

- ☐ Créer un compte GitHub avec un nom d'utilisateur professionnel (par exemple `prenom-nom`).
- ☐ Activer la double authentification sur GitHub.
- ☐ Installer GitHub Desktop et s'y connecter avec son compte.
- ☐ Installer un éditeur de texte, par exemple Visual Studio Code ou sa version entièrement libre VSCodium.

- [ ] Avec son équipe : terminer le backlog et le tableau Trello.

Atelier 2 – Collaborer sur GitHub

> **Lundi 16 novembre**
>
> L'atelier 2 commence le **lundi 16 novembre**, après le cours sur Git et GitHub. Ne le commencez pas avant.

7 Le dépôt GitHub de l'équipe

Dépôt · collaborateurs

1. Le Scrum Master crée sur GitHub un dépôt **privé** nommé `1gest-nom-du-projet`, avec un fichier `README.md`.
2. Dans **Settings** → **Collaborators**, il invite tous les membres et **Chris-CHEVALIER**.
3. Chaque membre accepte l'invitation puis clone le dépôt avec GitHub Desktop.
4. Le Scrum Master crée le dossier `docs/` avec un fichier vide nommé `need.md` (il sera complété à l'étape 9), puis fait le premier commit et le pousse. Les autres récupèrent ce commit.

Ne mettez jamais dans le dépôt un mot de passe, une clé d'accès ou une donnée personnelle (téléphone, adresse). Même privé, un dépôt peut être partagé ou fuiter.

> **Pour valider, assurez-vous que…**
>
> - le dépôt est privé, avec toute l'équipe et moi comme collaborateurs ;
> - le dépôt est cloné sur la machine de chaque membre.

8 Les issues et le suivi du projet

Issues · Milestones · GitHub Projects

Votre backlog quitte Trello pour GitHub. Les tâches vivent alors au même endroit que les documents de l'équipe.

1. Créez deux **milestones** (jalons) : « Sprint 1 » et « Sprint 2 », avec leur date de fin.
2. Créez une **issue par user story** des deux sprints : la phrase de la story, ses critères d'acceptation sous forme de cases à cocher, son milestone, un label de priorité et ses points dans le titre.
3. Créez un **projet** GitHub (onglet **Projects**) relié au dépôt et ajoutez-y toutes les issues.
4. Vue **Board** : les colonnes Todo, In Progress, Done jouent le rôle de votre tableau Kanban.

5. Vue **Roadmap** : ajoutez une date de début et une date de fin à chaque issue. Vous obtenez votre diagramme de Gantt.

Ajoutez enfin **une issue par document** de l'étape 9, assignée à son auteur. Ce sont les tâches d'aujourd'hui.

> **Pour valider, assurez-vous que…**
>
> - chaque issue des deux sprints a son milestone ;
> - les vues Board et Roadmap de votre projet GitHub sont renseignées.

9 Les documents de l'équipe

Markdown · branches · pull requests

Chaque document est écrit en **Markdown** par un auteur, sur **sa propre branche**, puis proposé dans une **pull request**. Un autre membre la relit et l'approuve avant la fusion dans `main`. Mettez `Closes #numéro` dans la description pour fermer l'issue du document automatiquement.

Fichier	Contenu
`docs/need.md`	Le besoin reformulé : objectif SMART, QQOQCCP, utilisateurs, livrables, jalons, contraintes et hors périmètre.
`docs/user-stories.md`	Toutes les user stories avec critères d'acceptation, priorité et points.
`docs/planning.md`	Les deux sprints : capacité, stories retenues, total des points, jalons datés, ce qui reste dans le backlog.
`docs/retrospective.md`	La rétrospective de l'étape 11.
`README.md`	La page d'accueil du dépôt. Elle suit le modèle ci-dessous.

Chaque membre est **auteur d'au moins une pull request** et **relecteur d'au moins une autre**. Des noms de branches clairs, par exemple `docs/need` ou `docs/planning`.

Le README de l'équipe

Le README est la première chose qu'on lit en ouvrant votre dépôt. Il est évalué à part dans la grille. Il contient, dans cet ordre :

1. un seul titre `#` : le nom du projet, suivi de deux phrases de présentation ;
2. `## Le client et son besoin` : le besoin en trois phrases maximum et un lien vers `docs/need.md` ;
3. `## L'équipe` : un **tableau** Markdown avec chaque membre et son rôle ;
4. `## Liens` : le tableau Trello et le projet GitHub ;

5. `## Documents` : un lien vers chaque fichier de `docs/` ;
6. `## Avancement` : des **cases à cocher** (`- [x]` fait, `- [ ]` à faire) pour les grandes étapes du projet ;
7. `## Usage de l'IA` : quel outil et pour quoi faire. Sinon, « Aucun outil d'IA utilisé ».

Vérifiez le rendu sur github.com : aucun symbole Markdown ne doit apparaître tel quel (un `**` visible, un tableau affiché en texte brut…).

Un modèle de README à adapter

README.md

```
# Nom du projet

Deux phrases : ce que fait l'application et pour qui.

## Le client et son besoin

Le besoin du client en trois phrases maximum.
Détails : [docs/need.md](docs/need.md)

## L'équipe

| Membre | Rôle          |
|--------|---------------|
| Prénom | Client        |
| Prénom | Product Owner |
| Prénom | Scrum Master  |

## Liens

- [Tableau Trello](https://trello.com/b/…)
- [Projet GitHub](https://github.com/…/projects/1)

## Documents

- [Besoin reformulé](docs/need.md)
- [User stories](docs/user-stories.md)
- [Planning des sprints](docs/planning.md)
- [Rétrospective](docs/retrospective.md)

## Avancement

- [x] Besoin reformulé et validé par le client
- [x] Backlog estimé et planifié
- [ ] Rétrospective rédigée

## Usage de l'IA

Outil utilisé, pour quoi faire (ou « Aucun outil d'IA utilisé »).
```

Le Markdown en 30 secondes

docs/planning.md

```
# Planning

## Sprint 1 (30 novembre → 11 décembre)

Capacité : **20 points**

- [ ] Inscription des joueurs (5)
- [ ] Grille des courses (8)
- [ ] Connexion des arbitres (5)

Total : 18 points · Jalon : démonstration au client le 11 décembre
```

`#` fait un titre, `-` une puce, `**…**` du gras, `- [ ]` une case à cocher, `[texte](lien)` un lien. Laissez une ligne vide avant une liste ou un tableau. GitHub affiche le résultat mis en forme. Toute la syntaxe.

Pour valider, assurez-vous que…

- les cinq documents sont dans `main` ;
- chaque pull request a été relue et approuvée par un autre membre et ferme son issue.

10 Le conflit volontaire

Conflit · résolution

Un conflit arrive tôt ou tard. Mieux vaut le provoquer une première fois, à froid.

1. Deux membres créent chacun une branche à partir de `main`, à jour.
2. Chacun modifie **la même ligne** du `README.md` : le slogan de l'équipe, chacun le sien.
3. Chacun ouvre sa pull request. La première est fusionnée.
4. La seconde affiche alors un conflit. Le second membre met sa branche à jour avec `main` dans GitHub Desktop (**Branch** → **Update from main**), puis ouvre le `README.md` dans VS Code. Ensemble, choisissez avec les boutons au-dessus du conflit : **Accept Current Change**, **Accept Incoming Change** ou **Accept Both Changes**. Enregistrez, faites le commit puis poussez.
5. Fusionnez la seconde pull request.

Pour valider, assurez-vous que…

- la pull request qui a eu un conflit est résolue puis fusionnée ;
- le README final ne contient plus de marqueur de conflit.

11 La rétrospective

Amélioration continue

Prenez 15 minutes pour regarder votre façon de travailler pendant ces deux journées, pas votre produit. Le Scrum Master anime. Chacun donne au moins une idée par colonne.

▶ Commencer	■ Arrêter	↻ Continuer
Ce qu'on devrait se mettre à faire	Ce qui nous a fait perdre du temps	Ce qui a bien marché

Terminez par **deux actions concrètes**, chacune avec un responsable, que vous appliqueriez au prochain sprint. Le tout va dans `docs/retrospective.md`, par une pull request comme les autres.

★ La soutenance et la grille

Lundi 16 novembre · à partir de 14h

10 minutes par équipe. Le groupe de Clermont-Ferrand présente sur Teams en partageant son écran.

1. **5 minutes de présentation**, chaque membre prend la parole :
   - le client présente son besoin initial en une phrase ;
   - l'équipe montre le besoin reformulé et ce qui est hors périmètre ;
   - le backlog, les priorités et une estimation qui a fait débat ;
   - les deux sprints et les jalons, sur la vue Roadmap ;
   - une pull request relue et le conflit résolu ;
   - les deux actions de la rétrospective.
2. **Une question à chaque membre**, sans IA ni notes. Elle porte sur une notion du cours appliquée à votre projet ou sur un document de l'équipe.

Seul ce qui est poussé dans le dépôt avant votre passage est évalué. Je relis le tableau Trello et le dépôt après les soutenances.

Grille d'évaluation (sur 20)

Critère	Ce que je vérifie	Points
Livrables du groupe		
Besoin reformulé	`docs/need.md` : objectif SMART, QQOQCCP, livrables, jalons, contraintes, hors périmètre	2
Backlog	10 stories ou plus, bien formées, avec critères d'acceptation et priorité MoSCoW	2
Estimation et planification	Points cohérents, deux sprints de 20 points maximum, Must d'abord, jalons datés	2
Tableau Trello	Listes, cartes complètes, état cohérent avec le reste du projet	1
Suivi sur GitHub	Issues, milestones, vues Board et Roadmap renseignées	1
Collaboration avec Git	Une branche et une pull request par document, relues, chacun auteur et relecteur, conflit résolu	2
Rétrospective	Trois colonnes renseignées, deux actions avec un responsable	1
README	Les sept sections du modèle, tableau de l'équipe, liens qui fonctionnent, cases d'avancement, Markdown correctement rendu sur GitHub	1
Soutenance		
Présentation	Structure respectée, 5 minutes tenues, chacun prend la parole (note commune)	4
Question individuelle	Réponse complète (4), partielle (2), absente ou fausse (0) (note individuelle)	4
Total		**20**

Un membre qui n'a contribué à aucun document (aucune pull request ni relecture) peut voir sa note de livrables réduite.



Avant la soutenance

- ☐ Le tableau Trello est à jour et je suis invité.
- ☐ Le dépôt privé contient les cinq documents dans `main` et je suis collaborateur.

- ☐ Chaque membre a ouvert au moins une pull request et en a relu au moins une autre.
- ☐ Les issues ont un milestone. Les vues Board et Roadmap sont renseignées.
- ☐ Le conflit volontaire a été résolu et le README ne contient plus de marqueur.
- ☐ Le README suit le modèle : sept sections, tableau de l'équipe, liens qui fonctionnent, déclaration d'usage de l'IA. Son rendu sur GitHub est propre.
- ☐ Chaque membre sait expliquer chaque document de l'équipe.
